

# NET2008 DevOps: Practical Assessment 1

Log Triage Tool | Python project | 5% of final grade

## The situation

You are on call. A server room produces thousands of log lines an hour and nobody can read them all. Build a small command-line tool that reads a log file, tells you what is in it, and flags the incidents that need a human. You decide how to build it: how many functions, which classes, how to organise the code. Everything from Weeks 3 and 4 is available to you.

## What you build

One Python file, **PA1\_<your-last-name>.py**, started like this:

```
python PA1_Moeini.py LOGFILE
```

It reads LOGFILE, writes a report to `report.json` in the current folder, and prints a short summary of your choice to the screen. Start from an empty file. There is no template.

## The log format

```
<timestamp> [<LEVEL>] <component>: <message>
2026-09-16T08:42:11Z [WARN ] auth: user "kadmin" from 10.10.4.12 failed login (attempt 3/12)
```

- timestamp: YYYY-MM-DDTHH:MM:SSZ. LEVEL is one of DEBUG INFO WARN ERROR CRIT, written in square brackets and padded with a space when short ([WARN ]).
- component: one word, then a colon and a space. message: the rest of the line, never empty.
- Blank lines are ignored. Any other line that does not follow the format is **malformed**: record it and carry on. A bad line must never stop the program.

## What report.json must contain

| Key         | Value                                                                                                                                                                                                                                               |
|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| total_lines | Number of non-blank lines in the file (valid and malformed).                                                                                                                                                                                        |
| malformed   | List of the line numbers of malformed lines. Lines are numbered from 1 and blank lines count for numbering.                                                                                                                                         |
| levels      | Object with all five levels as keys (DEBUG to CRIT) and how many valid lines each has, including 0.                                                                                                                                                 |
| components  | Object: component name to the number of valid lines it wrote.                                                                                                                                                                                       |
| ips         | Every distinct valid IPv4 address found in valid messages, sorted numerically (10.2.0.1 before 10.10.0.1). A valid address is four numbers from 0 to 255 separated by dots, without leading zeros: 300.1.1.1, 1.2.3 and 01.2.3.4 are not addresses. |
| incidents   | List of the incidents you detect (next section).                                                                                                                                                                                                    |

## Incidents

Your tool must detect two kinds of incident in valid lines. Each becomes one object in the incidents list:

| type         | A line is this incident when                                          | Extra field                   | Example line                                              |
|--------------|-----------------------------------------------------------------------|-------------------------------|-----------------------------------------------------------|
| failed_login | component is auth and the message contains failed login (attempt N/M) | attempts = N (a whole number) | ... auth: user x from 1.2.3.4 failed login (attempt 7/12) |

| type      | A line is this incident when                                                  | Extra field                  | Example line                                           |
|-----------|-------------------------------------------------------------------------------|------------------------------|--------------------------------------------------------|
| link_down | component is net and the message starts with link down and contains for N min | minutes = N (a whole number) | ... net: link down on Gi0/2 for 45 min (peer 10.2.2.2) |

Every incident object has type, line (its line number), its extra field, and a severity: a whole number from 1 (minor) to 5 (critical). **You choose the severity rule**, but a larger attempts or minutes must never give a lower severity than a smaller one. If the number in the message is not a whole number written with digits (for example attempt x/12), the line is not reported as an incident.

## Example

Input file:

```
2026-09-16T08:00:04Z [INFO ] web: GET /health 200 from 10.2.0.7
2026-09-16T08:00:19Z [WARN ] auth: user "kadmin" from 10.10.4.12 failed login (attempt 7/12)
this line is garbage

2026-09-16T08:01:02Z [ERROR] net: link down on Gi0/2 for 45 min (peer 10.2.2.2)
2026-09-16T08:01:30Z [DEBUG] web: cache refresh, bad address 300.1.1.1 ignored
```

A correct report.json (your severity values may differ; layout and spacing may differ):

```
{
  "total_lines": 5,
  "malformed": [3],
  "levels": {"DEBUG": 1, "INFO": 1, "WARN": 1, "ERROR": 1, "CRIT": 0},
  "components": {"web": 2, "auth": 1, "net": 1},
  "ips": ["10.2.0.7", "10.2.2.2", "10.10.4.12"],
  "incidents": [
    {"type": "failed_login", "line": 2, "attempts": 7, "severity": 3},
    {"type": "link_down", "line": 5, "minutes": 45, "severity": 2}
  ]
}
```

## Behaviour

- A normal run exits with code 0.
- If LOGFILE does not exist: print a message that includes the file name and exit with code 1. No traceback.
- If the program is not given exactly one argument: print a line starting with usage: and exit with code 2.
- Empty files, files with only bad lines, and large files (tens of thousands of lines) must all work. Finish within a few seconds.

## Show these techniques

Your code must show the following techniques, and your design should include them naturally:

- **A custom exception** class that you raise and catch (for example for a malformed line).
- **A class hierarchy**: a base class with at least two subclasses that each override a method or property (for example, the two incident kinds).
- **A decorator** that you write and use (for example, to count how often something runs, or to time it).
- **File handling** with with or pathlib, and no bare except:.
- **Clean code**: only standard library modules (sys os json re pathlib collections functools datetime itertools math argparse time random), no eval/exec/global, nothing running at import except definitions and a if \_\_name\_\_ == "\_\_main\_\_": block, and a docstring on at least 80% of

your functions and classes.

## Marks

| Area                                                                                        | Marks      |
|---------------------------------------------------------------------------------------------|------------|
| Runs end to end and writes a valid report.json                                              | 10         |
| Report is correct: total_lines 5, malformed 10, levels 8, components 8, ips 10              | 41         |
| Incidents detected correctly (type, line, attempts or minutes) 14, severity rule sensible 5 | 19         |
| Command-line behaviour and error handling                                                   | 10         |
| The five techniques above (3 each)                                                          | 15         |
| Readability and structure (marked by the instructor)                                        | 5          |
| <b>Total</b>                                                                                | <b>100</b> |

Your program will be run on many other log files, including empty files and files full of bad lines. Do not tune your code to one file.

## Sample data

**PA1\_sample.log** has 56 lines with a few bad ones. A correct tool reports 55 lines, malformed lines [10, 30, 40, 50], 51 valid entries, levels DEBUG 7, INFO 18, WARN 14, ERROR 5, CRIT 7, 10 addresses (first 10.1.3.99, last 203.36.141.16) and 20 incidents (10 failed\_login, 10 link\_down). Make your own small log files for the edge cases.

## Rules and submission

- Work alone and write your own code. Sharing or copying code is an academic integrity offence.
- Put your name and student ID in a comment at the top. Submit only **PA1\_<your-last-name>.py** to **Practical Assessment 1 Submission**. Not a notebook, not a ZIP, not report.json.
- Late submissions receive a grade of 0.

## Where to start

Get one line parsed. Then a whole file. Then the counts, the addresses, and the incidents. Test after every step on PA1\_sample.log and on tiny files you write yourself. Add the classes, the exception, and the decorator when your design asks for them, not at the end.